TECHNICAL ASSESSMENT · WEEK 2

Unit Converter Web App

Built in VS Code with Claude Code — Development Documentation

HTML5 CSS3 Vanilla JavaScript Single File

Prepared for: Coaching review

Author: Jeffrey De La Cruz Barrera

Date: July 14, 2026

Tooling: Visual Studio Code + Claude Code (Sonnet 5)

1. Project Overview

This assignment involved building a distance unit converter as a self-contained web application, developed inside VS Code using Claude Code as an AI pair-programming assistant. The project was completed in two iterations within a single chat session: an initial minimal version, followed by a significantly expanded, production-quality rebuild driven by a detailed design and feature brief.

The final deliverable is a single `index.html` file — no external frameworks, libraries, or build tools — that converts distances between kilometers and miles. It layers on smart input parsing, a swap control, real-world scale context, and clipboard integration on top of the core conversion logic.

> **Conversion factor used:** 1 mile = 1.60934 km. Kilometers→Miles divides by this factor; Miles→Kilometers multiplies by it.

2. Development Timeline (This Session)

1. **Initial build — three-file version**

   Created a basic converter split across `index.html`, `styles.css`, and `script.js`: a numeric input, a dropdown to choose Km→Mi or Mi→Km, a Convert button, and a result display, with light/dark-aware styling and basic input validation.

2. **Verification**

   Opened the app in the default browser to confirm both conversion directions and the invalid-input message worked as expected.

3. **Full redesign brief**

   Received a detailed specification calling for a single-file, glassmorphism "space-dark" themed rebuild with smart unit-detecting input, a swap control, a "cosmic scale" real-world context engine, and clipboard integration.

4. **Rebuild**

   Removed the old `styles.css` / `script.js` files and consolidated everything into one `index.html` with inline `<style>` and `<script>` blocks implementing all requested features.

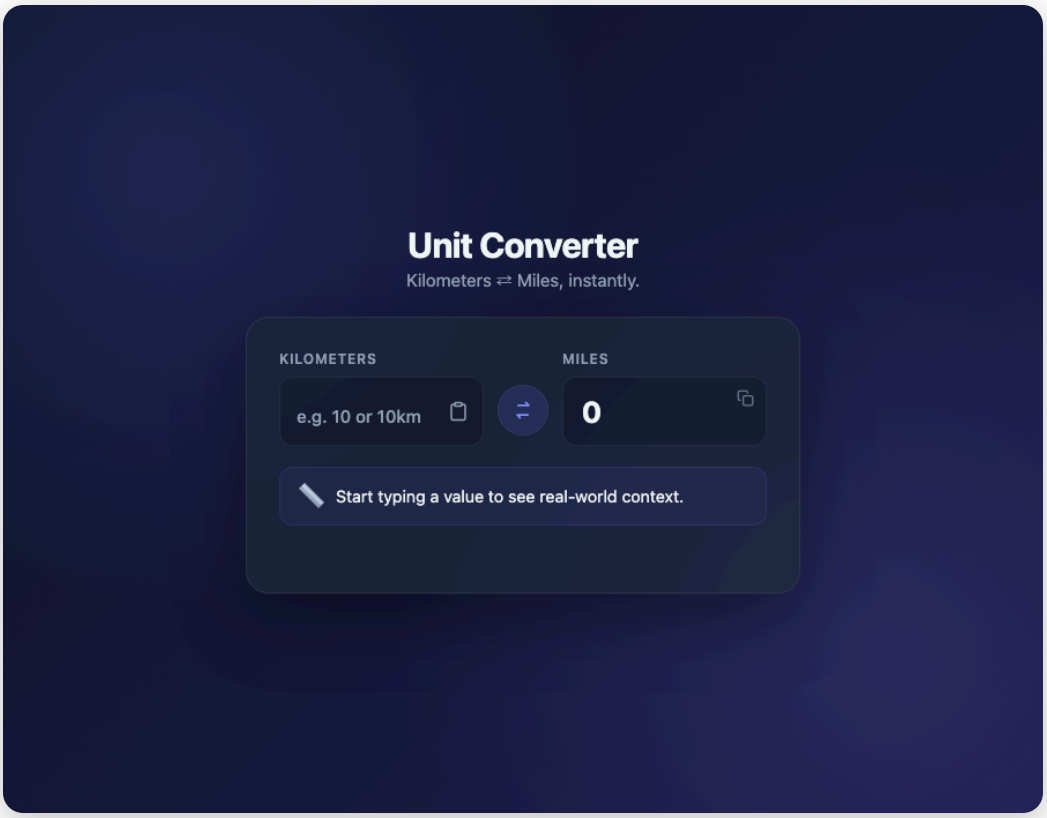
5. **Verification**

   Re-opened the app in the browser and manually verified smart parsing (e.g. "10", "5mi"), the swap button, the scale-context engine across all four distance bands, click-to-copy, and negative/invalid input handling.

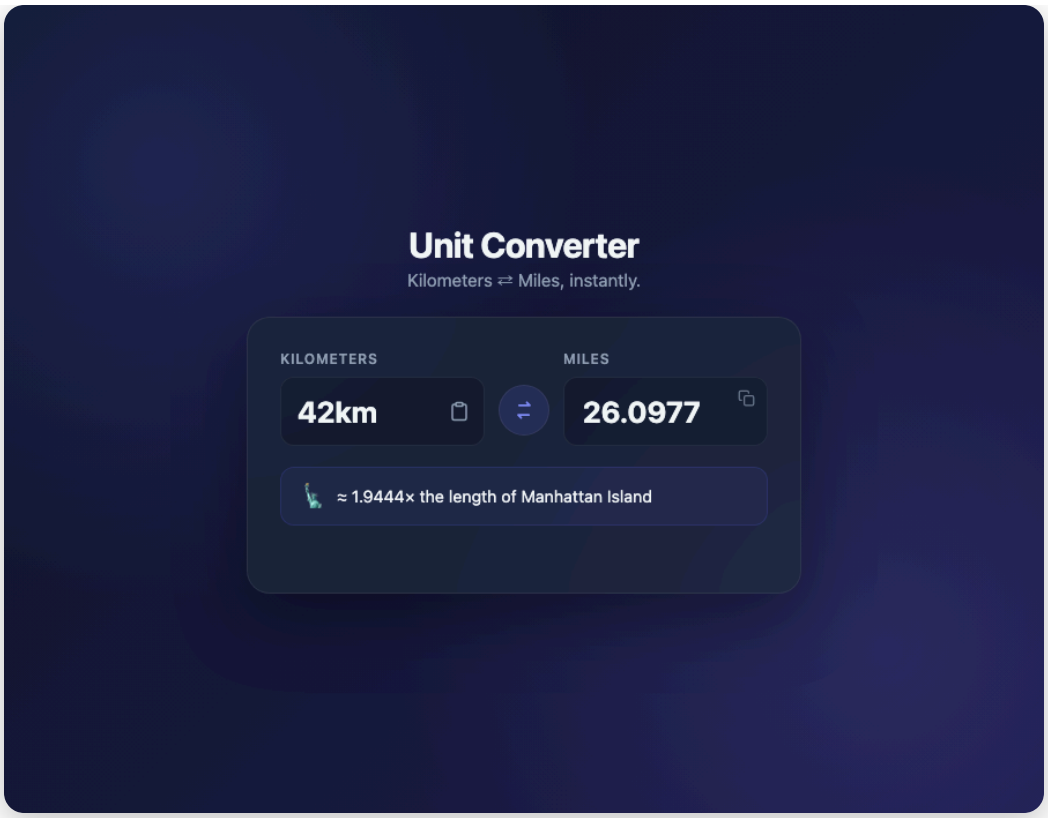
6. **Documentation**

   Generated this PDF report — capturing the project overview, full source code, and a feature-by-feature breakdown — for coaching review.

3. Application in Use



Default state — empty input, prompting the user to enter a value.



Smart-detected input "42km" auto-converted to 26.0977 mi, with the Cosmic Scale Context Engine showing the Manhattan Island comparison for the 10–100 km band.

4. Features Implemented

4.1 Core Conversion Engine

Feature	Description
Bidirectional conversion	Converts kilometers → miles and miles → kilometers using the standard factor 1 mi = 1.60934 km.
Live recalculation	Result updates on every keystroke, batched via `requestAnimationFrame` so rapid typing only triggers one render per frame.
Number formatting	Results are rounded to a maximum of 4 decimal places with trailing zeros stripped, and large numbers get thousand separators.

4.2 Smart Bidirectional Input

Feature	Description
Unit auto-detection	A regex parser (`/^([0-9.]+)\s*([a-zA-Z]*)$/`) reads the typed value. Typing "5mi" or "10km" automatically switches the active conversion direction and updates both labels — no dropdown needed.
Single input field	One field drives the whole interface; direction can also be set via the dedicated swap control.

4.3 Dynamic Swap Control

Feature	Description
Direction flip	A circular button with an animated SVG icon reverses the conversion direction (Km→Mi ⇄ Mi→Km).
Continuity	On swap, the previous output value is carried into the input field so the conversion feels continuous instead of resetting.
Micro-interaction	The icon performs a quick rotate animation on click, and focus returns to the input automatically.

4.4 Cosmic Scale Context Engine

Below the result, a context card translates the distance into a relatable real-world comparison, chosen by scale:

Range	Context shown
< 1 km	Equivalent in meters plus an estimated walking step count (0.762 m average stride).
1 km – 10 km	Number of laps around a standard 400 m athletics track.
10 km – 100 km	Multiples of the length of Manhattan Island (≈ 21.6 km).
> 100 km	Percentage of Earth's equatorial circumference (≈ 40,075 km).

4.5 Clipboard Integration

Feature	Description
Copy result	Clicking the output block copies the formatted result and unit to the clipboard and shows a temporary "Copied to clipboard!" toast.
Paste into input	A clipboard icon inside the input field reads clipboard text via the Clipboard API and inserts it directly, re-running the conversion.
Graceful fallback	Falls back to a hidden-textarea `execCommand('copy')` approach if the modern Clipboard API is unavailable, and hides the paste button entirely on browsers without clipboard read support.

4.6 UI / UX Design

Feature	Description
Space-dark theme	Deep slate-to-navy gradient background with soft indigo/violet glow accents.
Glassmorphism card	Semi-transparent card with `backdrop-filter: blur(16px)` and a subtle border.
Responsive layout	Mobile-first design; the input/output row stacks vertically below 480px, with the swap icon rotating to match.
Accessibility	ARIA live regions for results and errors, keyboard-operable copy action (Enter/Space), and a `prefers-reduced-motion` override.
Micro-interactions	Buttons scale down slightly on `:active`, inputs show a glowing focus ring, and the swap icon animates on click.

4.7 Edge-Case Handling

Case	Behavior

Empty input	Output resets to "0"; context card shows a neutral prompt.
Negative numbers	Rejected with a clear inline message ("Distance can't be negative.") without breaking execution.
Invalid text	Unparseable input (e.g. stray symbols, unrecognized unit suffixes) shows "Enter a valid number, e.g. 10 or 10km."
Zero	Handled as a valid value with its own context message ("Zero distance — no movement at all.").

5. Technical Notes

- **Performance** DOM elements are queried once at load and cached in variables; no repeated `getElementById` calls inside hot paths.
- **Performance** Input handling is batched per animation frame via `requestAnimationFrame` rather than recalculating on every synchronous keystroke event.
- **Architecture** All logic is wrapped in an IIFE with `'use strict'` to avoid polluting the global scope.
- **Portability** Zero external dependencies — the file can be opened directly in any modern browser with no build step or server required.

6. Full Source Code — index.html

The complete, final contents of the single-file application delivered for this assignment.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Unit Converter — km ⇄ mi</title>
<meta name="description" content="Smart, single-file kilometer/mile converter with real-world scale
context.">

<style>
  /* ============================================================
     DESIGN TOKENS — space-dark theme, glassmorphism accents
     ============================================================ */
  :root {
    --bg-grad-start: #0f172a;
    --bg-grad-end: #1e1b4b;
    --card-bg: rgba(30, 41, 59, 0.7);
    --card-border: rgba(255, 255, 255, 0.08);
    --text-primary: #f1f5f9;
    --text-muted: #94a3b8;
    --accent: #6366f1;
    --accent-bright: #818cf8;
    --accent-glow: rgba(99, 102, 241, 0.45);
    --error: #f87171;
    --success: #34d399;
    --radius-lg: 20px;
    --radius-md: 12px;
  }

  * {
    box-sizing: border-box;
  }

  html, body {
    height: 100%;
  }

  body {
    margin: 0;
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 24px;
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
    color: var(--text-primary);
    background: linear-gradient(135deg, var(--bg-grad-start) 0%, var(--bg-grad-end) 100%);
    /* Fixed cosmic glow accents — pure CSS, no canvas/JS cost */
    background-image:
      radial-gradient(circle at 15% 20%, rgba(99, 102, 241, 0.18), transparent 40%),
      radial-gradient(circle at 85% 80%, rgba(129, 140, 248, 0.15), transparent 45%),
```

```
      linear-gradient(135deg, var(--bg-grad-start) 0%, var(--bg-grad-end) 100%);
    background-attachment: fixed;
  }

  /* ============================================================
     LAYOUT SHELL
     ============================================================ */
  .app {
    width: 100%;
    max-width: 480px;
  }

  header {
    text-align: center;
    margin-bottom: 20px;
  }

  header h1 {
    margin: 0 0 4px;
    font-size: clamp(1.5rem, 4vw, 1.9rem);
    font-weight: 700;
    letter-spacing: -0.02em;
  }

  header .tagline {
    margin: 0;
    color: var(--text-muted);
    font-size: 0.95rem;
  }

  .card {
    background: var(--card-bg);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid var(--card-border);
    border-radius: var(--radius-lg);
    padding: 28px;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.35);
  }

  /* ============================================================
     INPUT / OUTPUT ROW
     ============================================================ */
  .io-row {
    display: flex;
    align-items: stretch;
    gap: 12px;
  }

  .io-block {
    flex: 1;
    min-width: 0;
    display: flex;
    flex-direction: column;
    gap: 8px;
  }

  .io-block label {
    font-size: 0.75rem;
```

```
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  color: var(--text-muted);
}

.input-wrap {
  position: relative;
  display: flex;
  align-items: center;
}

#value-input {
  width: 100%;
  padding: 14px 42px 14px 14px;
  font-size: clamp(1.3rem, 5vw, 1.6rem);
  font-weight: 700;
  font-family: inherit;
  color: var(--text-primary);
  background: rgba(15, 23, 42, 0.55);
  border: 1px solid var(--card-border);
  border-radius: var(--radius-md);
  outline: none;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

#value-input::placeholder {
  color: var(--text-muted);
  font-weight: 500;
  font-size: 0.95rem;
}

#value-input:focus {
  border-color: var(--accent);
  box-shadow: 0 0 0 4px var(--accent-glow);
}

.icon-btn {
  position: absolute;
  right: 6px;
  display: flex;
  align-items: center;
  justify-content: center;
  width: 32px;
  height: 32px;
  padding: 0;
  border: none;
  border-radius: 8px;
  background: transparent;
  color: var(--text-muted);
  cursor: pointer;
  transition: background 0.15s ease, color 0.15s ease, transform 0.1s ease;
}

.icon-btn:hover {
  background: rgba(255, 255, 255, 0.06);
  color: var(--accent-bright);
}
```

```
.icon-btn:active {
  transform: scale(0.95);
}

.icon-btn svg {
  width: 18px;
  height: 18px;
}

/* Swap button — sits between the two io-blocks */
.swap-btn {
  align-self: center;
  flex: none;
  width: 44px;
  height: 44px;
  margin-top: 20px; /* visually aligns with input/output fields, below labels */
  display: flex;
  align-items: center;
  justify-content: center;
  border: 1px solid var(--card-border);
  border-radius: 50%;
  background: rgba(99, 102, 241, 0.15);
  color: var(--accent-bright);
  cursor: pointer;
  transition: background 0.15s ease, box-shadow 0.2s ease, transform 0.1s ease;
}

.swap-btn:hover {
  background: rgba(99, 102, 241, 0.3);
  box-shadow: 0 0 16px var(--accent-glow);
}

.swap-btn:active {
  transform: scale(0.9);
}

.swap-btn svg {
  width: 20px;
  height: 20px;
}

.swap-btn.spin svg {
  animation: spin-rotate 0.4s ease;
}

@keyframes spin-rotate {
  from { transform: rotate(0deg); }
  to   { transform: rotate(180deg); }
}

/* Output block */
.output-block {
  position: relative;
  justify-content: flex-start;
  padding: 14px 16px;
  background: rgba(15, 23, 42, 0.55);
  border: 1px solid var(--card-border);
  border-radius: var(--radius-md);
  cursor: pointer;
```

```
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

.output-block:hover,
.output-block:focus-visible {
  border-color: var(--accent);
  box-shadow: 0 0 0 4px var(--accent-glow);
  outline: none;
}

.output-block:active {
  transform: scale(0.98);
}

.output-value {
  font-size: clamp(1.3rem, 5vw, 1.6rem);
  font-weight: 700;
  line-height: 1.2;
  word-break: break-word;
}

.copy-hint-icon {
  position: absolute;
  top: 10px;
  right: 10px;
  width: 15px;
  height: 15px;
  color: var(--text-muted);
  opacity: 0.7;
}

/* ============================================================
   CONTEXT CARD — cosmic scale engine output
   ============================================================ */
.context-card {
  display: flex;
  align-items: center;
  gap: 10px;
  margin-top: 18px;
  padding: 14px 16px;
  background: rgba(99, 102, 241, 0.08);
  border: 1px solid rgba(99, 102, 241, 0.18);
  border-radius: var(--radius-md);
  font-size: 0.9rem;
  color: var(--text-muted);
  transition: opacity 0.2s ease;
}

.context-icon {
  flex: none;
  font-size: 1.3rem;
  line-height: 1;
}

.context-text {
  color: var(--text-primary);
}

/* ============================================================
```

```
   ERROR MESSAGE
   ============================================================ */
.error-msg {
  min-height: 18px;
  margin-top: 12px;
  font-size: 0.85rem;
  font-weight: 500;
  color: var(--error);
  text-align: center;
}

/* ============================================================
   TOAST
   ============================================================ */
.toast {
  position: fixed;
  left: 50%;
  bottom: 32px;
  transform: translate(-50%, 16px);
  padding: 10px 18px;
  background: rgba(15, 23, 42, 0.95);
  border: 1px solid var(--card-border);
  border-radius: 999px;
  font-size: 0.85rem;
  font-weight: 600;
  color: var(--success);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.25s ease, transform 0.25s ease;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);
}

.toast.show {
  opacity: 1;
  transform: translate(-50%, 0);
}

.toast.toast-error {
  color: var(--error);
}

/* ============================================================
   RESPONSIVE — stack vertically on narrow viewports
   ============================================================ */
@media (max-width: 480px) {
  .card {
    padding: 20px;
  }

  .io-row {
    flex-direction: column;
  }

  .swap-btn {
    align-self: center;
    margin: -2px 0;
    transform: rotate(90deg);
  }
```

```
    .swap-btn.spin svg {
      animation: spin-rotate-vert 0.4s ease;
    }
  }

  @keyframes spin-rotate-vert {
    from { transform: rotate(0deg); }
    to   { transform: rotate(180deg); }
  }

  @media (prefers-reduced-motion: reduce) {
    * {
      animation-duration: 0.001ms !important;
      transition-duration: 0.001ms !important;
    }
  }
</style>
</head>
<body>
  <div class="app">
    <header>
      <h1>Unit Converter</h1>
      <p class="tagline">Kilometers ⇄ Miles, instantly.</p>
    </header>

    <main class="card">
      <div class="io-row">
        <!-- INPUT -->
        <div class="io-block">
          <label for="value-input" id="input-label">Kilometers</label>
          <div class="input-wrap">
            <input
              id="value-input"
              type="text"
              inputmode="decimal"
              autocomplete="off"
              spellcheck="false"
              placeholder="e.g. 10 or 10km"
              aria-label="Value to convert"
            >
            <button class="icon-btn" id="paste-btn" type="button" title="Paste from clipboard" aria-
label="Paste from clipboard">
              <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-
linecap="round" stroke-linejoin="round">
                <rect x="8" y="2" width="8" height="4" rx="1"></rect>
                <path d="M16 4h2a2 2 0 0 1 2 2v14a2 2 0 0 1-2 2H6a2 2 0 0 1-2-2V6a2 2 0 0 1 2-2h2">
</path>
              </svg>
            </button>
          </div>
        </div>

        <!-- SWAP -->
        <button class="swap-btn" id="swap-btn" type="button" aria-label="Swap conversion direction">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2" stroke-
linecap="round" stroke-linejoin="round">
            <path d="M7 7h11l-3-3"></path>
            <path d="M17 17H6l3 3"></path>
          </svg>
```

```
        </button>

        <!-- OUTPUT -->
        <div class="io-block">
          <label id="output-label">Miles</label>
          <div class="output-block" id="output-block" role="button" tabindex="0" aria-label="Copy
result to clipboard">
            <div class="output-value" id="output-value">0</div>
            <svg class="copy-hint-icon" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2" stroke-linecap="round" stroke-linejoin="round">
              <rect x="9" y="9" width="13" height="13" rx="2"></rect>
              <path d="M5 15H4a2 2 0 0 1-2-2V4a2 2 0 0 1 2-2h9a2 2 0 0 1 2 2v1"></path>
            </svg>
          </div>
        </div>
      </div>

      <div class="context-card" id="context-card">
        <span class="context-icon" id="context-icon">🔪</span>
        <span class="context-text" id="context-text">Start typing a value to see real-world context.
</span>
      </div>

      <div class="error-msg" id="error-msg" role="alert" aria-live="assertive"></div>
    </main>
  </div>

  <div class="toast" id="toast" role="status" aria-live="polite"></div>

<script>
(() => {
  'use strict';

  /* ============================================================
     CONSTANTS
     ============================================================ */
  const KM_PER_MILE = 1.60934;
  const TRACK_LAP_KM = 0.4;          // standard 400m athletics track
  const MANHATTAN_KM = 21.6;         // approx. length of Manhattan Island
  const EARTH_CIRCUMFERENCE_KM = 40075; // equatorial circumference
  const AVG_STEP_M = 0.762;          // average adult walking step length

  /* ============================================================
     DOM REFS (queried once — no repeated lookups in hot paths)
     ============================================================ */
  const input = document.getElementById('value-input');
  const pasteBtn = document.getElementById('paste-btn');
  const swapBtn = document.getElementById('swap-btn');
  const inputLabel = document.getElementById('input-label');
  const outputLabel = document.getElementById('output-label');
  const outputBlock = document.getElementById('output-block');
  const outputValueEl = document.getElementById('output-value');
  const contextCard = document.getElementById('context-card');
  const contextIcon = document.getElementById('context-icon');
  const contextText = document.getElementById('context-text');
  const errorMsg = document.getElementById('error-msg');
  const toast = document.getElementById('toast');

  /* ============================================================
```

```
   STATE
   ============================================================ */
let mode = 'km-to-mi';   // 'km-to-mi' | 'mi-to-km'
let lastResult = null;   // numeric result of the last successful conversion
let rafId = null;
let toastTimer = null;

/* ============================================================
   NUMBER FORMATTING
   Strips trailing zeros, caps at 4 decimal places, adds
   thousand separators for readability.
   ============================================================ */
function formatNumber(num) {
  if (!isFinite(num)) return '0';
  let rounded = Math.round((num + Number.EPSILON) * 10000) / 10000;
  if (Object.is(rounded, -0)) rounded = 0;
  return rounded.toLocaleString('en-US', { maximumFractionDigits: 4 });
}

// Plain (non-localized) numeric string, used when feeding a value
// back into the input field so it remains re-parseable.
function formatPlain(num) {
  let rounded = Math.round((num + Number.EPSILON) * 10000) / 10000;
  if (Object.is(rounded, -0)) rounded = 0;
  return rounded.toString();
}

/* ============================================================
   SMART INPUT PARSER
   Detects an optional trailing unit token ("km"/"mi") so users
   can type "10km" or "5 mi" and the app infers direction.
   ============================================================ */
function parseSmartInput(raw) {
  const trimmed = raw.trim();
  if (trimmed === '') return { status: 'empty' };
  if (trimmed.startsWith('-')) return { status: 'negative' };

  const match = trimmed.match(/^([0-9.]+)\s*([a-zA-Z]*)$/);
  if (!match) return { status: 'invalid' };

  const value = parseFloat(match[1]);
  if (!isFinite(value)) return { status: 'invalid' };

  const unitToken = match[2].toLowerCase();
  let unit = null;
  if (unitToken) {
    if (unitToken.startsWith('k')) unit = 'km';
    else if (unitToken.startsWith('mi')) unit = 'mi';
    else return { status: 'invalid' };
  }

  return { status: 'ok', value, unit };
}

/* ============================================================
   COSMIC SCALE CONTEXT ENGINE
   Translates a km-equivalent distance into relatable, everyday
   scale context.
   ============================================================ */
```

```
function getContext(km) {
  if (km === 0) {
    return { icon: '📍', text: 'Zero distance — no movement at all.' };
  }
  if (km < 1) {
    const meters = km * 1000;
    const steps = Math.round(meters / AVG_STEP_M);
    return {
      icon: '🚶',
      text: `≈ ${formatNumber(meters)} m — about ${steps.toLocaleString('en-US')} walking steps`
    };
  }
  if (km <= 10) {
    const laps = km / TRACK_LAP_KM;
    return {
      icon: '🏃',
      text: `≈ ${formatNumber(laps)} laps around a standard 400m athletics track`
    };
  }
  if (km <= 100) {
    const lengths = km / MANHATTAN_KM;
    return {
      icon: '🗽',
      text: `≈ ${formatNumber(lengths)}× the length of Manhattan Island`
    };
  }
  const pct = (km / EARTH_CIRCUMFERENCE_KM) * 100;
  return {
    icon: '🌍',
    text: `≈ ${formatNumber(pct)}% of Earth's equatorial circumference`
  };
}

/* ============================================================
   UI HELPERS
   ============================================================ */
function currentFromUnit() {
  return mode === 'km-to-mi' ? 'km' : 'mi';
}

function currentToUnitAbbrev() {
  return mode === 'km-to-mi' ? 'mi' : 'km';
}

function updateLabels() {
  if (mode === 'km-to-mi') {
    inputLabel.textContent = 'Kilometers';
    outputLabel.textContent = 'Miles';
    input.placeholder = 'e.g. 10 or 10km';
  } else {
    inputLabel.textContent = 'Miles';
    outputLabel.textContent = 'Kilometers';
    input.placeholder = 'e.g. 10 or 10mi';
  }
}

function setOutput(result) {
  outputValueEl.textContent = result === null ? '0' : formatNumber(result);
}
```

```
function setContext(kmValue) {
  if (kmValue === null) {
    contextIcon.textContent = '📏';
    contextText.textContent = 'Start typing a value to see real-world context.';
    return;
  }
  const ctx = getContext(kmValue);
  contextIcon.textContent = ctx.icon;
  contextText.textContent = ctx.text;
}

function showError(msg) {
  errorMsg.textContent = msg;
}

function clearError() {
  errorMsg.textContent = '';
}

/* ============================================================
   CORE CONVERSION / RENDER CYCLE
   ============================================================ */
function recalc() {
  const parsed = parseSmartInput(input.value);
  clearError();

  if (parsed.status === 'empty') {
    lastResult = null;
    setOutput(null);
    setContext(null);
    return;
  }

  if (parsed.status === 'negative') {
    lastResult = null;
    setOutput(null);
    setContext(null);
    showError("Distance can't be negative.");
    return;
  }

  if (parsed.status === 'invalid') {
    lastResult = null;
    setOutput(null);
    setContext(null);
    showError('Enter a valid number, e.g. 10 or 10km.');
    return;
  }

  // Auto-detect direction from a typed unit suffix (e.g. "5mi").
  if (parsed.unit && parsed.unit !== currentFromUnit()) {
    mode = parsed.unit === 'km' ? 'km-to-mi' : 'mi-to-km';
    updateLabels();
  }

  const value = parsed.value;
  const result = mode === 'km-to-mi' ? value / KM_PER_MILE : value * KM_PER_MILE;
  const kmEquivalent = mode === 'km-to-mi' ? value : value * KM_PER_MILE;
```

```
  lastResult = result;
  setOutput(result);
  setContext(kmEquivalent);
}

// Batch rapid keystrokes into a single render per animation frame.
function scheduleRecalc() {
  if (rafId) cancelAnimationFrame(rafId);
  rafId = requestAnimationFrame(recalc);
}

/* ============================================================
   CLIPBOARD (with graceful fallback for older/restrictive browsers)
   ============================================================ */
async function copyText(text) {
  try {
    if (navigator.clipboard && navigator.clipboard.writeText) {
      await navigator.clipboard.writeText(text);
      return true;
    }
    throw new Error('Clipboard API unavailable');
  } catch (err) {
    try {
      const ta = document.createElement('textarea');
      ta.value = text;
      ta.style.position = 'fixed';
      ta.style.opacity = '0';
      document.body.appendChild(ta);
      ta.select();
      document.execCommand('copy');
      document.body.removeChild(ta);
      return true;
    } catch (fallbackErr) {
      return false;
    }
  }
}

function showToast(message, isError) {
  toast.textContent = message;
  toast.classList.toggle('toast-error', !!isError);
  toast.classList.add('show');
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => toast.classList.remove('show'), 1800);
}

async function handleCopyOutput() {
  if (lastResult === null || !isFinite(lastResult)) return;
  const text = `${formatNumber(lastResult)} ${currentToUnitAbbrev()}`;
  const ok = await copyText(text);
  showToast(ok ? 'Copied to clipboard!' : 'Unable to copy', !ok);
}

async function handlePaste() {
  try {
    const text = await navigator.clipboard.readText();
    input.value = text;
    recalc();
```

```
      input.focus();
    } catch (err) {
      showToast('Clipboard access denied', true);
    }
  }

  /* ============================================================
     SWAP DIRECTION
     ============================================================ */
  function swapDirection() {
    mode = mode === 'km-to-mi' ? 'mi-to-km' : 'km-to-mi';
    updateLabels();

    // Carry the previous result forward as the new input, so the
    // conversion feels continuous rather than resetting to blank.
    if (lastResult !== null && isFinite(lastResult)) {
      input.value = formatPlain(lastResult);
    }

    recalc();
    input.focus();

    swapBtn.classList.remove('spin');
    // Force reflow so the animation can retrigger on rapid clicks.
    void swapBtn.offsetWidth;
    swapBtn.classList.add('spin');
  }

  /* ============================================================
     EVENT WIRING
     ============================================================ */
  input.addEventListener('input', scheduleRecalc, { passive: true });

  swapBtn.addEventListener('click', swapDirection);

  outputBlock.addEventListener('click', handleCopyOutput);
  outputBlock.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' || e.key === ' ') {
      e.preventDefault();
      handleCopyOutput();
    }
  });

  pasteBtn.addEventListener('click', handlePaste);

  // Hide the paste affordance entirely if the browser can't support it,
  // rather than offering a control that will always fail.
  if (!navigator.clipboard || !navigator.clipboard.readText) {
    pasteBtn.style.display = 'none';
  }

  /* ============================================================
     INIT
     ============================================================ */
  updateLabels();
  setOutput(null);
  setContext(null);
})();
</script>
```

```
</body>
</html>
```